

Kinesis Design Document

William N. Johnson

Version 0.5.3 (in development), October 2026

Abstract

Kinesis is a desktop application for keeping and studying the videos you learn from. It saves YouTube videos with their transcripts, lets you summarize, tag, file and link them, and lets you plan them into routines such as a fitness program or a school course. This document describes how the application is put together: its architecture, its data model, the main features and the reasons behind the less obvious decisions. It is written for people who will work on the code.

Contents

1	What Kinesis is for	3
1.1	Goals	3
1.2	Non-goals	3
2	Architecture	3
2.1	The window	5
3	Workspaces and storage	6
3.1	Workspaces	6
3.2	The database	7
3.3	Main tables	8
3.4	Attachments	8
4	The Drive system	8
4.1	Two spellings	9
4.2	Homes and links	9
4.3	Changing the tree	9
4.4	Sequences	9
5	Getting videos in	9
6	Search and filtering	10
7	The sidebar	10

8	Keeping up with YouTube	11
8.1	Renamed channels	11
8.2	Renamed and removed videos	12
8.3	Orphaned videos	12
9	Links between things	12
10	Glossary and Biography	13
10.1	Biographies	14
11	AI summaries and images	15
12	The Calendar	15
12.1	Tracks	15
12.2	The week	16
12.3	Time windows	16
12.4	Starting over	17
12.5	Deleted videos	17
12.6	Export	17
13	Pins	18
14	Graph view	18
15	Deleting and the Trash	19
16	Permissions	20
17	Sharing and export	20
17.1	Kinpak	20
17.2	Obsidian	21
18	Sync	21
19	Interface conventions	21
20	Testing	22
21	Open work	22

1 What Kinesis is for

Most people who learn from video keep track of it badly. A useful lecture ends up in a browser bookmark, the point they wanted to remember is somewhere in minute 40, and the connection between that lecture and an article they read last month exists only in their head. Kinesis treats a video as a document. Once saved, it has a transcript you can search, a summary you can edit, tags and glossary terms, notes, attached files, and links to and from other videos, terms and people.

The second half of the application is about doing something with that library over time. Tracks on the Calendar turn a list of videos and tasks into a plan: one item after another, a weekly schedule like a class, or a repeating rotation like a workout split.

1.1 Goals

- Everything lives on the user’s machine, in a SQLite database they can copy, back up or move to an external drive. No account is needed to use the app.
- A workspace can be handed to someone else as a single file, without leaking API keys or local paths.
- The person who sets up a workspace (a teacher, a research lead) can decide what its readers see and can change.
- Large libraries stay usable. The maintainer’s own library has more than eight thousand items, and features are tested against that size.

1.2 Non-goals

- Kinesis does not host or download video. Playback is YouTube’s embedded player.
- Sync clients never upload. A sync server publishes curated content; it does not collect anything back.
- The project is in beta. The sync protocol and the Kinpak format may change between versions, and there are deliberately no compatibility shims for older formats.

2 Architecture

Kinesis is a Tauri 2 application. The backend is Rust (edition 2021, minimum Rust 1.88) and the user interface is React 19, built with Vite 7 and styled with Tailwind CSS 4. The window has no native title bar; the app draws its own.

Layer	Responsibility
<code>src/</code> (TypeScript)	Every screen, plus <code>api.ts</code> , the one place that calls backend commands. A large feature keeps its parts in a folder of its own (<code>src/components/calendar</code>), common controls live in <code>src/components/ui</code> , and self-contained feature logic (pins, bulk assign, returning to the graph) lives in hooks under <code>src/hooks</code> .
<code>src-tauri/src/commands</code>	Tauri commands. Each runs its database work on a blocking thread so a slow query never freezes the window.
<code>src-tauri/src/db</code>	All SQL, one module per area (videos, glossary, calendar, pins and so on).
<code>src-tauri/src/youtube</code> , <code>venice.rs</code>	Talking to YouTube and the AI provider.
<code>src-tauri/src/sync</code>	The sync client engine.
<code>crates/kinesis-sync-proto</code>	Wire types, the settings allowlist, hashing and the pack codec, shared by the client, the reference server and the Kinpak format.
<code>crates/kinesis-sync-server</code>	The reference sync server.

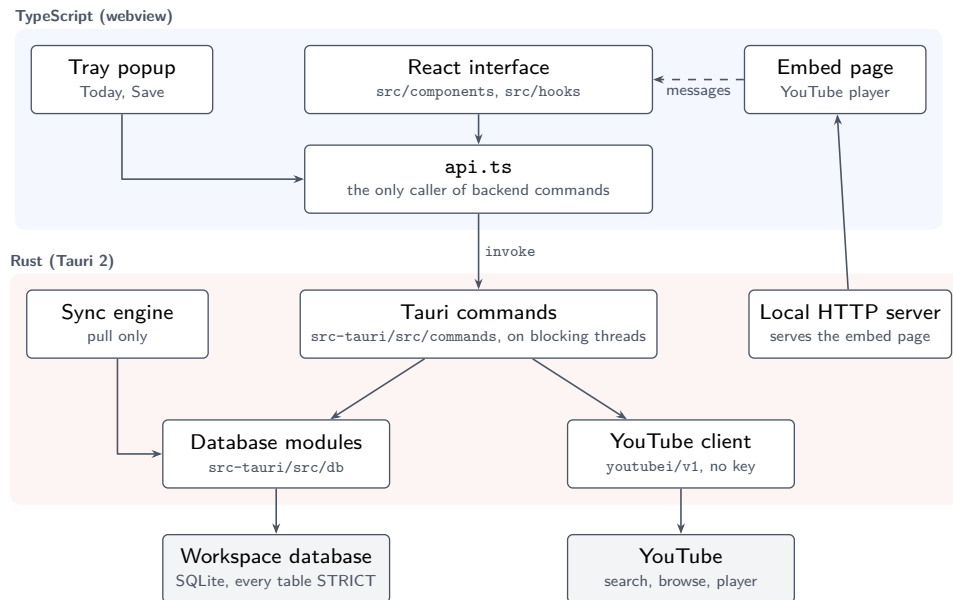


Figure 1: How the parts talk to each other. Everything the interface asks of the backend goes through `api.ts`; the embed page answers the sidebar with window messages (a seek, the current time, a video that played to its end).

A few smaller pieces run beside the main window:

- A small local HTTP server serves the YouTube embed page on `localhost`. The video ID it is given is checked against YouTube’s 11-character alphabet before it is placed in the page, which is what keeps it from being an injection point.
- On Windows, a tray icon opens a small frameless popup with two tabs. *Today* lists the Calendar’s items for today, and anything overdue, with checkboxes. *Save* takes a YouTube

link, filled in from the clipboard, and saves it to the Library. The popup opens on Today unless a new link is on the clipboard. With *Keep running in the tray* on, closing the main window hides it instead of quitting.

- Every way of leaving the app (the close button, Alt+F4, the tray's Quit) first asks the page whether something is unsaved. A page that does not answer within a short timeout does not keep the app open.
- Window size, position, layout and navigation style are per machine, stored in a JSON file in the config directory. Everything else is per workspace, in that workspace's database.

2.1 The window

Figure 2 shows the main window on the Library. The title bar **1** holds the section tabs (a workspace can rename them) and, on the right, the pin menu **2**, whose number counts the Calendar items due today. The search bar **3** turns typed prefixes into facet chips (Section 6). The Drive panel **4** filters the grid to one branch of the tree. A card whose thumbnail comes back as YouTube's gray placeholder **5** has most likely been removed from YouTube; Section 8 explains what the app does about it. Every section ends in a status bar **6**; the Library's shows the result count, the Orphaned and Trash chips, and the sort controls.

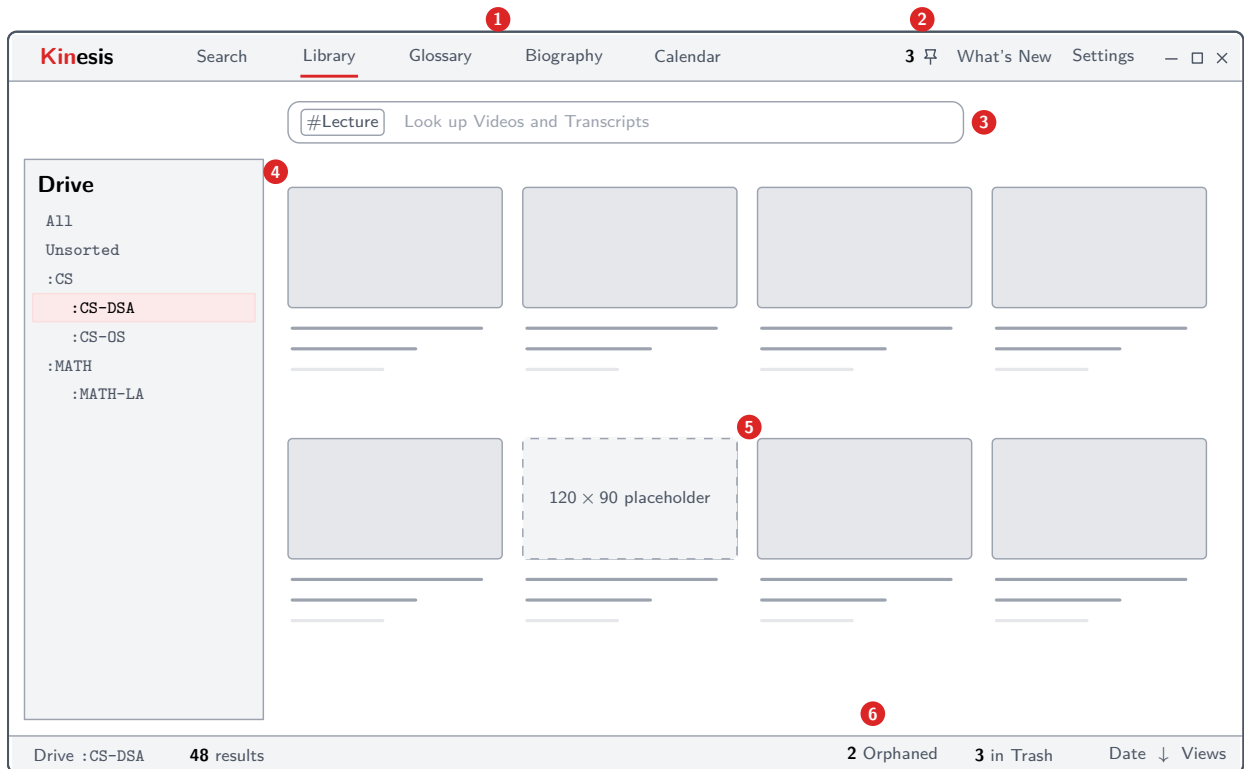


Figure 2: The main window on the Library, with a Drive selected and a facet chip in the search bar.

3 Workspaces and storage

3.1 Workspaces

A workspace is a folder under the application's data directory. It holds an `init.conf` and usually the database, `kinesis_data.db`. If the database lives somewhere else, for example on an external drive, the folder holds only the `init.conf`, which points at it:

```
kinesisapp/  
  Computer Science Workspace/  
    kinesis_data.db  
    init.conf db_path: E:\Drives\CS (only when the database lives elsewhere)  
    last_opened: 1789000000000
```

There is no registry of workspaces. The recent list is a scan of these folders, and the most recent workspace is the one with the newest `last_opened`. A workspace whose drive is unplugged stays in the list, dimmed, with a *Locate* action. An exclusive lock file beside the database stops two Kinesis windows from opening the same workspace.

The Workspaces screen (Figure 3) is that scan made visible. The open workspace is marked with the accent bar **1**; one on an unplugged drive shows *Not found* and *Locate* **2**. The main area offers the three ways to add a workspace **3**: start an empty one, import a Kinpak, or point at a folder that already holds a database. It is also the first screen on a fresh install, so *About Kinesis* **4** opens a short walkthrough of the app. Behind it runs Conway's Game of Life, which a click on the background seeds.

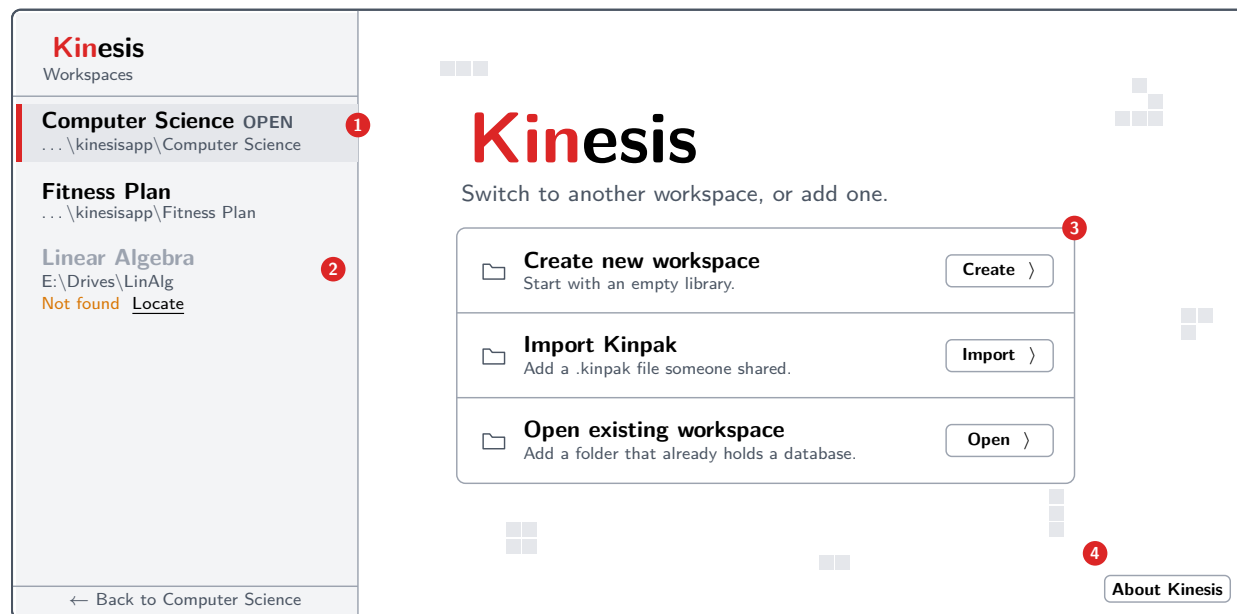


Figure 3: The Workspaces screen, with the open workspace, one whose drive is unplugged, and the ways to add one.

3.2 The database

Each workspace is one SQLite database, accessed through `rusqlite` with SQLite bundled. Three rules shape the schema:

1. Every table is declared `STRICT`. A value of the wrong type is an error instead of being quietly converted. This caught real bugs, such as a view count that came back from YouTube as "1,234".
2. There are no foreign keys. Cleanup happens in triggers: deleting a track deletes its entries and their completions, deleting a video removes it from Drive sequences and pins, and so on. The production database is maintained by hand outside the app, and triggers fit that workflow better than foreign key enforcement.
3. The production schema is checked in as `production_schema.sql`. A test opens a database built from it, runs `init_db` twice and asserts that nothing was added or removed. Any new table, column or trigger has to be added to that file, or the test fails.

New columns arrive through a small migration step: if `column_exists` says a column is missing, an `ALTER TABLE ... ADD COLUMN` adds it. Triggers that depend on new columns are created after that step, since an older database may not have the column yet.

3.3 Main tables

Table	Holds
Videos	One row per saved video: title, author, handle, length, transcript, summary, view count, publish date, comma-separated tags, home Drive (WDBS), date added, and search tokens.
ftsVideos	The FTS5 index over title, summary, tokens and Drive.
tblWDBS	The Drive tree: every node, with its alias, icon, color and markers.
VideoWDBSLinks	“Also in” links that file a video under more Drives than its home.
DriveSequence	Ordered watch lists, one per Drive.
Glossary	Quick Tags and Terms. One row per definition, with the Drives it is filed under.
Biographies	People and channels: display name, the reader’s alias for it, bio, social links, channel ID, subscriber count.
VideoNotes, VideoAttachments, AttachmentBlobs	Notes and files attached to videos.
Calendar*	Tracks, entries, completions, in-progress marks, terms, holiday sets and rules (Section 12).
PinnedVideos	The reader’s pinned videos.
OrphanedVideos	Saved videos YouTube no longer has, with what YouTube said and when it was found.
Settings, WorkspaceLabels	Settings and feature flags; the workspace name and section names.
SearchHistory, SyncItems	Search history; which rows a sync server owns.

3.4 Attachments

Files attached to a video are stored inside the database and addressed by content. **AttachmentBlobs** keeps each distinct file once, keyed by the SHA-256 of its original bytes, and **VideoAttachments** points videos at blobs. The same PDF attached to ten videos is stored once. Compressible types are deflated, but only when the result is smaller and only after decompressing it proves it gives back the original bytes exactly. Every read checks the bytes against the hash. A file must be under 128 MiB. Web links are stored as attachments of a special type whose bytes are the address, so they travel with everything else attachments already do.

4 The Drive system

Videos are filed in a tree called the Drive, written as paths such as **:CS-DSA** (Computer Science, then Data Structures and Algorithms). Internally it is called WDBS, for Workspace Drive-Based

Structure. A workspace can rename the section (a course workspace might call it “Modules”), so code and comments say “Drive” and the interface shows whatever name the workspace uses. The Manage Drive window is the exception and always says “Drive”.

4.1 Two spellings

A path has a display form and a storage form:

Display form	Storage form
:CS-DSA	$\theta\psi$ CS_DSA

The storage form is what the production database uses in `Videos.WDBS` and `VideoWDBSLinks`. The display form is used in `tblWDBS`, sequences and glossary filing. Conversion happens at the edges, and code that touches both must be careful about which one it holds.

4.2 Homes and links

Every video has one home in the tree, or none (“Unsorted”). It can also be linked into other Drives, which is how a lecture on hash tables appears under both `:CS-DSA` and `:INTERVIEW`. The production database has a trigger that rejects a home path that is not registered in `tblWDBS`, so the app registers a path and every level above it before filing a video there.

4.3 Changing the tree

Renaming, moving or merging a Drive is one operation, `relocate_drive`. A path is referenced from six places, and all six are updated together in one transaction: `tblWDBS`, `Videos.WDBS`, `VideoWDBSLinks`, `DriveSequence`, the Drives a glossary row is filed under, and `kinesis://drive/...` links inside text.

4.4 Sequences

A sequence is an ordered list of videos for one Drive, shown in the sidebar as First, Previous and Next. A sequence may include any video filed at or below its Drive, so `:CS` can walk across `:CS-DSA` and `:CS-OS`. The same video can sit in the sequences of several Drives, in different positions. Positions are kept as 1 to N on every write; reads only depend on their order, so a gap left by a deleted video does no harm.

5 Getting videos in

The Search section finds videos on YouTube through the same internal endpoints the YouTube website uses (`youtubei/v1/search`, `browse` and `player`), so searching needs no API key. Pasting a video, playlist or channel link, or typing a `@handle`, turns into a filter chip automatically.

Transcripts come from the caption tracks the player lists. When YouTube answers with “Sign in to confirm you’re not a bot”, the app treats it as YouTube limiting the network rather than a problem with the video, because asking again later often works.

A YouTube Data API key is optional. With one, the app fetches the channel ID and subscriber count the first time it sees a channel and stores them on the channel’s biography. The channel ID is captured at that moment because, unlike a handle, it cannot be recovered later if it was never saved. A missing subscriber count is stored as -1 , which is never a real count.

Saving a video computes its search tokens: the distinct, lowercased, punctuation-stripped words of the transcript that are not stop words. The original SQL for this asks SQLite for the character at each position, which gets slower the further into the text it goes. That is fine for one video and takes minutes for an import of thousands, so imports use a Rust version that reproduces the SQL’s rules exactly, quirks included. A test compares the two.

6 Search and filtering

Library search runs on SQLite’s FTS5. Each typed word becomes a prefix match. Facets narrow the search and are typed as prefixes that turn into a chip:

Typed	Meaning
#name	Videos with this Quick Tag
^name	Videos with this Term
@handle	Videos from this channel handle
@@name	Videos from a channel by display name or alias
>id	One video by its ID
!#	Videos with no tags at all
:CS	Videos in this Drive or below it

A ***** matches any run of characters, so ***harbor** finds “codeharbor”. Ghost text suggests the rest of a tag, term, handle, channel name or video ID as it is typed, and Tab accepts it. A channel’s alias is suggested after its real name, so **@@** finds a channel under either.

The Similar Videos tab in the sidebar ranks with BM25 over the same index, weighting title above summary above tokens (2.0, 1.5 and 1.0) and giving the Drive column no weight, so that two videos in the same folder are not called similar for that reason alone.

The Calendar’s search bar is a filter rather than a search. Plain words match an item’s name, description or track, and one facet at a time (**track:**, **priority:**, **status:**, **kind:**) narrows by that field. Cells holding a match are tinted instead of hiding everything else, so the week does not jump around while typing. Enter moves to the next matching day, up to a year away.

7 The sidebar

Opening a video opens the sidebar over whatever section is showing (Figure 4). It holds the player, the transcript and the AI summary (both editable as Markdown), the video’s tags and detected terms, its notes and attachments, its Drive home and links, Similar Videos, and Backlinks. The header has a pin button beside the close button ❶.

Under the header, a strip shows the Calendar tracks the video is on ❷: which track, its place in the list, its day, an *In progress* toggle and *Mark done*. When the video plays to its end, the

embed page says so and a window asks whether to mark it done on those tracks. It asks rather than marking the video, because people rewatch videos they have already finished.

The right-hand pane **3** switches between the transcript and the summary. Timestamps typed in square brackets, such as `[1:23]` **4**, become links that jump the player. The brackets are required so that a ratio like 2:1 or a clock time is not mistaken for one. A summary ends with a “Channel Info” footer **5** that links to the creator’s biography; when the reader has given the channel an alias, the alias shows as a badge in front of it. Editing guards against losing work: leaving the sidebar, switching videos or quitting with an unsaved edit asks first.

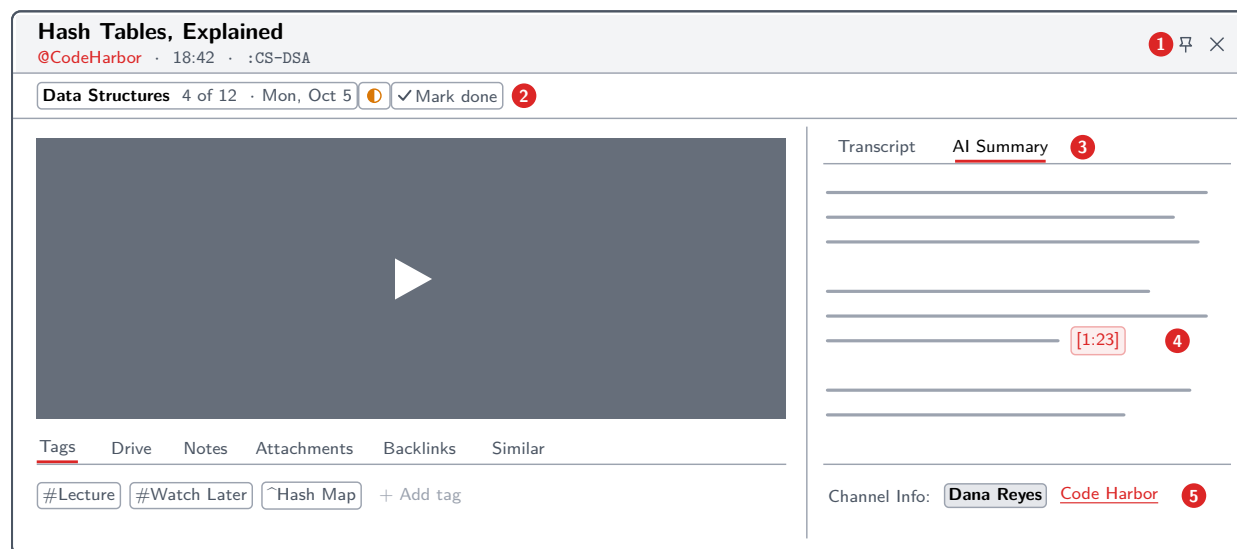


Figure 4: The sidebar. The track strip only appears when the video is on a Calendar track.

8 Keeping up with YouTube

A saved video is a snapshot. Channels rename themselves, change their handles, retitle videos and take videos down, and none of that reaches the library by itself. Kinesis checks the things a reader is looking at, through the same keyless endpoints it uses everywhere else.

8.1 Renamed channels

Opening a biography looks the channel up by its channel ID, which never changes, on the **browse** endpoint. The answer’s **channelMetadataRenderer** gives the channel’s current name and, from its vanity URL, its handle. The answer must carry the same channel ID, so a redirect to some other page is never taken for a rename. A biography saved before channel IDs were captured gets its ID from its handle the first time this runs.

A new handle is taken over at once: the biography’s row, its videos, its custom prompt, every **kinesis://bio/** link in any text, and the Channel Info footers of its videos. If another biography already has the new handle, nothing changes, since merging two people is not something to do without asking. A new display name is different, because the reader may know the channel by its old name. The window asks whether to keep the old name as the channel’s alias (Section 10.1);

either way the biography takes the new name. In a read-only workspace both are applied without asking, and no alias is made.

A stored name sometimes includes the handle, as in “Code Harbor (@CodeHarbor)”, because that is how some names arrive with a video. The comparison ignores a trailing handle in brackets, so such a name does not count as a rename.

8.2 Renamed and removed videos

Videos are checked on the **player** endpoint (Figure 5). Outside a browser YouTube refuses to play a video, but it still returns the title when the video exists. A removed video answers **ERROR** with no video details at all. That gives three outcomes:

- A title came back. The video exists; if the title changed, the library takes the new one. A video opened in the sidebar also shows a short *Video Title Renamed* notice with an OK button. There is nothing to choose, since a video has no alias.
- **ERROR**, or a private video. The video is marked orphaned. Its title is left alone, because there is no new one to take.
- Anything else, including “Sign in to confirm you’re not a bot”, an age gate, a region block or no answer at all. Nothing is concluded. A sign-in check is about the network asking, not about the video, and orphaning a video over it would be wrong.

Two things start a check. Opening a saved video in the sidebar checks it once per opening. The Library’s thumbnails are the other: YouTube serves a removed video’s thumbnail as a 404 with a 120 by 90 gray placeholder, where a real one is 480 by 360. A card whose thumbnail fails or comes back as the placeholder queues its video for a check, and so does a good thumbnail on a video that is currently orphaned, since the video may have come back. The thumbnail only starts the check; it never decides it, so a network hiccup cannot orphan a video. The queue runs one video at a time with a short pause between them and asks about each video at most once a session, so scrolling past a page of broken thumbnails does not flood YouTube.

8.3 Orphaned videos

An orphaned video stays in the library, because its transcript, summary, notes and links are still worth having. The **OrphanedVideos** table records it, and the Library’s status bar shows an *Orphaned* chip next to the Trash (Figure 2, ). The chip opens a list of orphaned videos, each of which can be opened or deleted, with a *Delete All*. Deleting goes through the Trash like any other delete. A video that YouTube answers for again is taken off the list.

Delete All is one bulk delete rather than a loop of single ones. The rows go in one transaction, and the links to all of the videos are removed in one pass over the stored texts, with each video’s own text changes recorded separately. Every video still becomes its own Trash item and can be restored by itself.

9 Links between things

Text anywhere in Kinesis can link to other things with Markdown links on a **kinesis://** address:

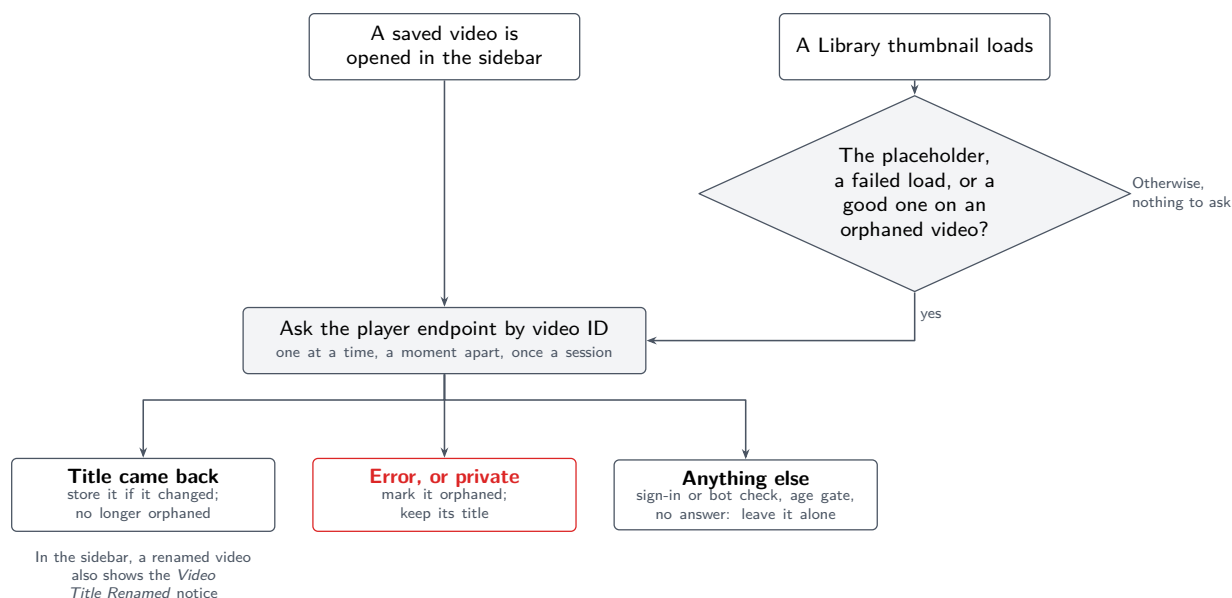


Figure 5: Checking a saved video against YouTube.

Address	Opens
<code>kinesis://video/ID?t=1:23</code>	A video, optionally at a moment
<code>kinesis://glossary/Term</code>	A glossary term
<code>kinesis://bio/handle</code>	A person's biography
<code>kinesis://drive/...</code>	A Drive in the Library
<code>kinesis://playlist/:CS-DSA</code>	Whatever is first in that Drive's sequence

Links can live in five places: video summaries, transcripts and notes, biographies, and glossary definitions. In a Markdown editor, typing @ or ^ followed by the start of a handle or term shows the rest as ghost text, and Enter or Tab turns it into a link.

Renaming or deleting a term, person or video rewrites every link to it. The scan looks only at texts containing `kinesis://`, which takes about a tenth of a second for the summaries of five thousand videos. Backlinks use the same scan in reverse. A summary's "Channel Info" footer links to the creator, but backlinks and the graph view ignore that footer: every summary has one, so a person's backlinks would otherwise just list all of their videos again.

10 Glossary and Biography

The Glossary holds two kinds of entries. A Quick Tag is a label with no definition. A Term has a definition, written in Markdown. A term can have a different definition in different Drives, so the table stores one row per definition with the list of Drives it is filed under. Terms can only be filed under top-level Drives.

Five Quick Tags are built in: Watch Later, Favorite, Revisit, Key Source and Follow Up. They cannot be renamed or deleted, and the app puts back any that are missing each time a workspace opens.

The Glossary can turn the terms on screen into flashcards (Figure 6): all of them, or one Drive's.

A card has the term on one side and the definition on the other, and *First* ❶ picks which side faces up. *Shuffle* mixes the deck and *Start over* returns to the first card ❷. Clicking the card, or pressing Space, turns it over with a 3D flip ❸; a definition keeps its links, so a term can lead straight to the video it came from. The arrows, or the arrow keys, move between cards ❹.

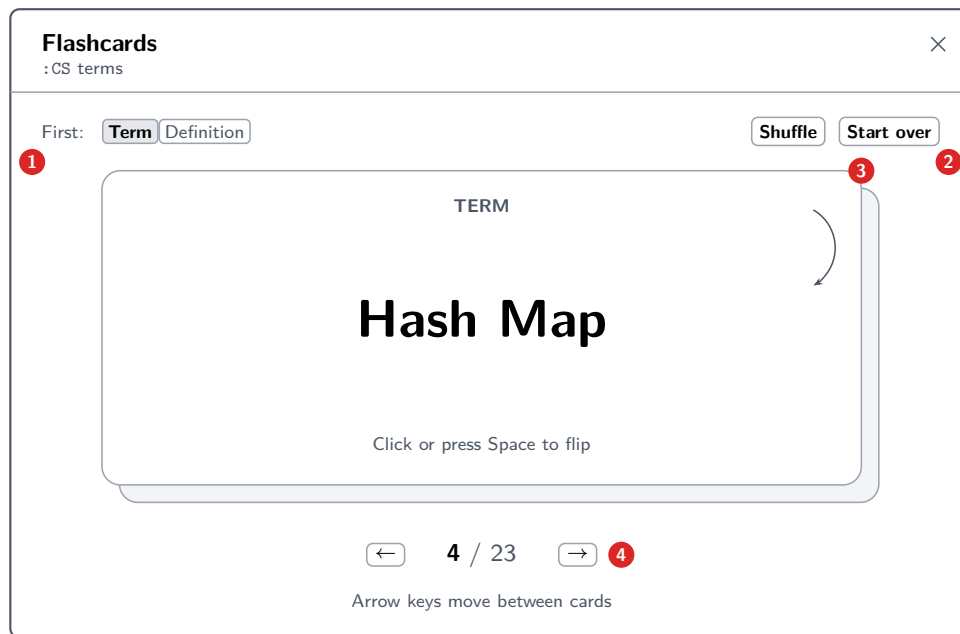


Figure 6: Flashcards for one Drive’s terms, showing the term side.

10.1 Biographies

A biography belongs to a channel handle. The app creates one the first time it saves a video from that channel, filling in the display name, and never overwrites a display name the user has set. Biographies carry a Markdown bio, social links and backlinks.

A biography can also have an alias: a name of the reader’s own for the channel, usually the person behind it. Channel names are often brand names, and “Code Harbor” says less to most readers than “Dana Reyes”, the person who hosts it. The alias is optional and is set in Edit Bio. Where it is set, it shows as a badge in front of the channel’s name (Figure 7, ❶), in the Biography list, in the biography’s window and in a summary’s Channel Info footer. @@ searches match it as well as the name, and the Obsidian export writes it as the note’s `aliases`, so a wiki link to the alias finds the note.

Edit Bio has the alias and the handle on two tabs, Alias and Handle Casing. The handle can only change its capitals, so @john can become @John but not @jean: the handle is the channel’s identity on YouTube, and a typo there would point the biography at someone else. The window refuses anything else before saving, and the backend checks again. A change of capitals is carried through like a rename from YouTube (Section 8), so videos, links and footers follow.

Show Alias, a switch in the Biography status bar ❷, is on by default. On, the badges show and the list is sorted by alias where there is one; off, the badges are hidden and the list goes by the channels’ own names. The biography’s own window always shows the badge. The setting is the reader’s, kept in the browser’s local storage.

When a channel has renamed itself on YouTube, opening its biography asks whether to keep the old name as the alias ³ (Section 8). That is the case the alias is most useful for: the reader still knows the channel by the name it used to have.

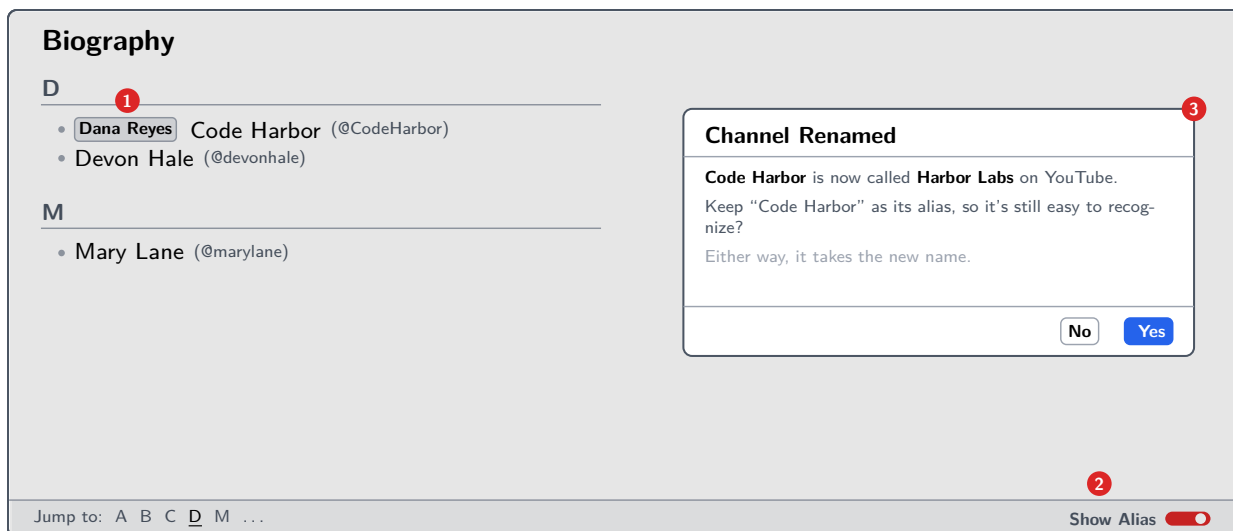


Figure 7: The Biography list with an alias badge, and the prompt shown for a channel that renamed itself.

11 AI summaries and images

Summaries are made with Venice. A workspace can store custom prompts per channel. After a summary is saved, the app appends a “Channel Info” footer naming the channel and linking it to its biography when one exists.

Summaries can also carry images, generated by Venice, taken from Pixabay, or uploaded. All of these features are optional and can be hidden by the workspace owner.

12 The Calendar

12.1 Tracks

A track is an ordered list of entries. An entry is a video or a task; a task has a name, an icon, a description and a priority from none to high. A video entry has the same description and priority, edited in the same window, but keeps the video’s title and icon. Each track advances in one of three ways:

- *One after another.* The first entry not yet done is up next. There are no dates.
- *On a schedule.* The track has weekdays and a start date, and the n th scheduled day belongs to the n th entry, whether or not it was done.
- *Schedule and finish.* Entries get days as above, but the track waits for each one to be done. An entry past its day and not done is overdue.

A track can loop, which makes it a rotation. Each time through the list is a cycle, and completions are stored per entry per cycle. The data model also lets a dated track be limited by terms (a semester, or a range that repeats every year) and skip or include holiday sets, with US federal holidays calculated rather than stored. These are used by the schedule, by imports and by the ICS export, but the app has no screen for editing them.

A track can also follow a Drive’s sequence. Its entries then come live from the sequence, and only entries with something of their own (a completion, a description) have a row.

The state of a track (which entry is up, how far along it is, what is overdue) is never stored. `resolve` works it out from the entries, completions and schedule each time it is asked. This keeps the stored data small and means there is nothing to fall out of date.

Items can be checked off in three places: the Calendar, the sidebar’s track chips, and the tray popup’s Today tab. The pin’s due-today count reflects all of them. Every call in `api.ts` that changes the Calendar announces it afterwards, as an app event that reaches every window, and whatever shows Calendar data reloads when it hears it. A check-off in the tray popup therefore shows in the main window straight away.

12.2 The week

Figure 8 shows the Calendar. The tracks panel on the left lists the tracks and, under *Up next* ❶, what is next on undated tracks and on dated tracks that have fallen behind. The week has a row per track and a column per day, with each entry on its day as a chip. A chip carries three kinds of mark:

- Done ❷: a check, with the name struck through.
- In progress ❸: a circle with its left half filled, in amber. It means started but not finished, and is set from the item’s menu, the track panel, or the sidebar’s track strip. It is stored per entry per cycle in `CalendarProgress`, and marking the entry done clears it. Overdue is not a separate state but a flag on top: an item can be in progress and overdue, and the `status:` facet finds it under both.
- Priority: a diamond, a gray outline for low, half filled yellow for medium, solid red for high. The fill grows with the level, so the levels can be told apart without seeing the colors.

While the search bar has a filter, cells holding a match are tinted ❹ and the rest of the week stays in place. Clicking a chip opens its menu: Mark Done, Mark In Progress, Open Video, and for a video Pin or Unpin.

12.3 Time windows

A dated track can have a time of day, such as 9:00 for 90 minutes. Two tracks clash when their times overlap, they share a weekday, and both are running at the same time. A track runs from its start date to the end of its last term, or forever if it has no terms or has a term that repeats yearly. The check runs inside the same transaction as the change that could cause a clash (creating, editing or unarchiving a track, or changing its terms), so a clash rolls the change back. Archived tracks are never checked.

Track names are unique among active tracks, ignoring case and repeated spaces. An archived track can share a name, so last year’s “Data Structures” can sit beside this year’s.

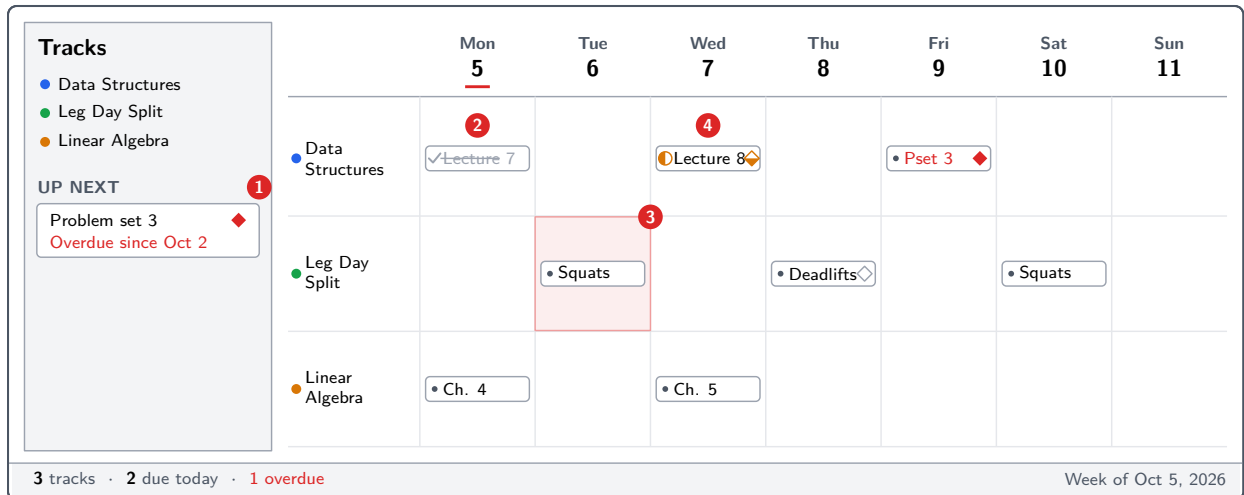


Figure 8: The Calendar’s week, with a done item, an item in progress, three priorities and a tinted search match.

12.4 Starting over

Start over sends a track back to its first entry in round 1, from today. Nothing is deleted. The track’s `cycle_offset` moves past every cycle used so far, so old completions remain as history under their own cycles and the new run starts clean. A dated track’s start date moves to today.

The previous start date and offset are saved, and the start over can be undone until something is marked done in the new run. Undoing after that would leave completions in a cycle the old run has not reached. Editing the start date by hand also clears the saved state, since undoing would then restore a date from before the edit.

12.5 Deleted videos

When a video is deleted, its entries stay in their tracks as tasks named after the video. The entry remembers the video’s ID in `former_video_id`, and a trigger on inserting into `Videos` reconnects it if the video comes back, through a Trash restore or by saving it again. A name the user typed by hand is kept through all of this.

12.6 Export

Dated tracks can be exported as an `.ics` file for Google Calendar, Apple Calendar or Outlook, for any period up to two years. The writer follows RFC 5545: text is escaped, lines are folded at 75 octets and end in CRLF. Every event has a stable UID made from the track, entry and cycle, so importing a newer export updates events instead of duplicating them. Tracks with a time window export timed events in floating local time; the rest export all-day events.

13 Pins

A pin marks a video the reader is working on now. The sidebar's pin button toggles it, and the pin menu in the title bar lists pinned videos, newest first, from any section. Under the list, a row shows how many Calendar items are due today, and a second row, shown only when tomorrow has anything, how many are to do tomorrow. Each opens the Calendar on that day's week with its items flashing briefly. Only today's count appears beside the pin icon; tomorrow's is a look ahead and is not added to it. Below that, each Essential tag opens a submenu of the videos carrying it. A video can also be pinned from its chip on the Calendar.

Figure 9 shows the menu open. The count beside the pin **1** is what is due today. Each pinned video **2** can be unpinned from the menu with the button that appears on hover. The Calendar rows **3** sit under the list, and the Essential tags **4** open their submenus to the left **5**, because the menu sits at the window's right edge. A submenu ends with *Show all in Library*, which opens the Library filtered to that tag.

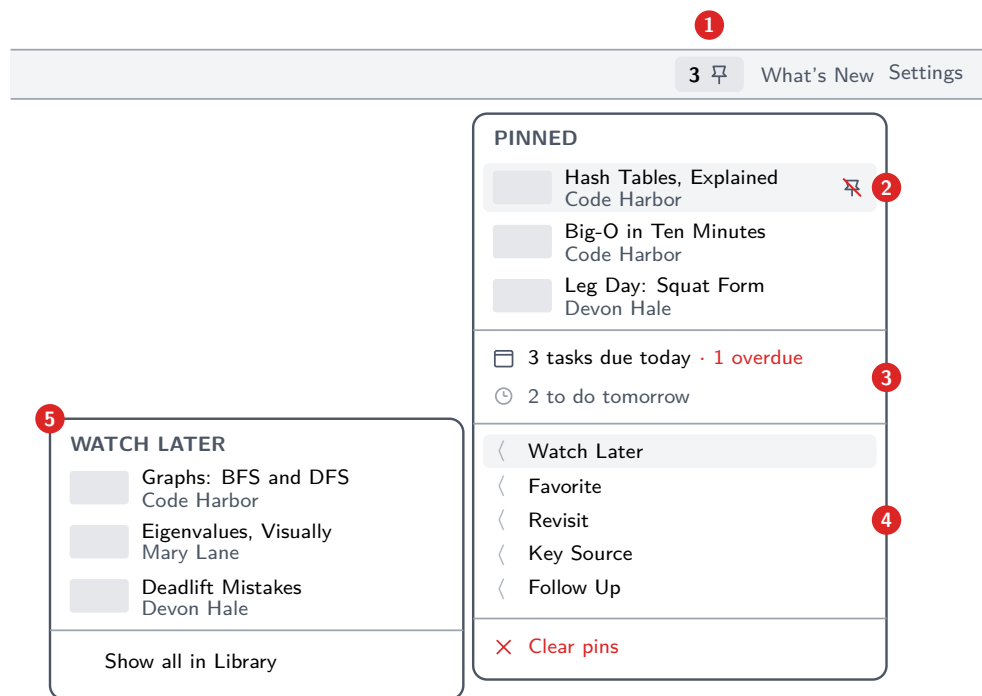


Figure 9: The pin menu with the Watch Later submenu open.

Pins have their own table rather than being a tag. A tag would travel in a Kinpak, would carry no time for ordering, and would show up as a chip on every pinned video. Pins are meant to stay a short list; there is no need to search for them.

14 Graph view

Opened from the command palette (Ctrl+K), the graph view draws every video, term and person as a dot, joined where one links to another. Lines from each video to its creator can be turned on, but are off by default because they outnumber the links people actually wrote.

The layout is a force simulation written for the app. Repulsion uses a Barnes-Hut quadtree stored in flat typed arrays, and the simulation runs in a Web Worker so panning and zooming stay smooth while it settles. On a test graph of 8,188 dots, a step takes about 14 to 17 ms and the layout settles in three to four seconds. Drawing batches all lines and dots into a handful of canvas paths and skips anything off screen.

Opening something from the graph and then closing it, or pressing Back, returns to the graph as it was left.

Figure 10 shows the window. Videos are gray, terms amber and people blue, and each kind can be hidden ❶. *Creator lines* and *Unconnected* ❷ add the lines to each video's creator and the dots with no links at all; both start off, since they mostly add noise. *Find* centers the first dot whose name matches, and *Fit* brings everything back into view ❸. Pointing at a dot highlights its lines and shows a card with its name and how many connections it has ❹; clicking opens it.

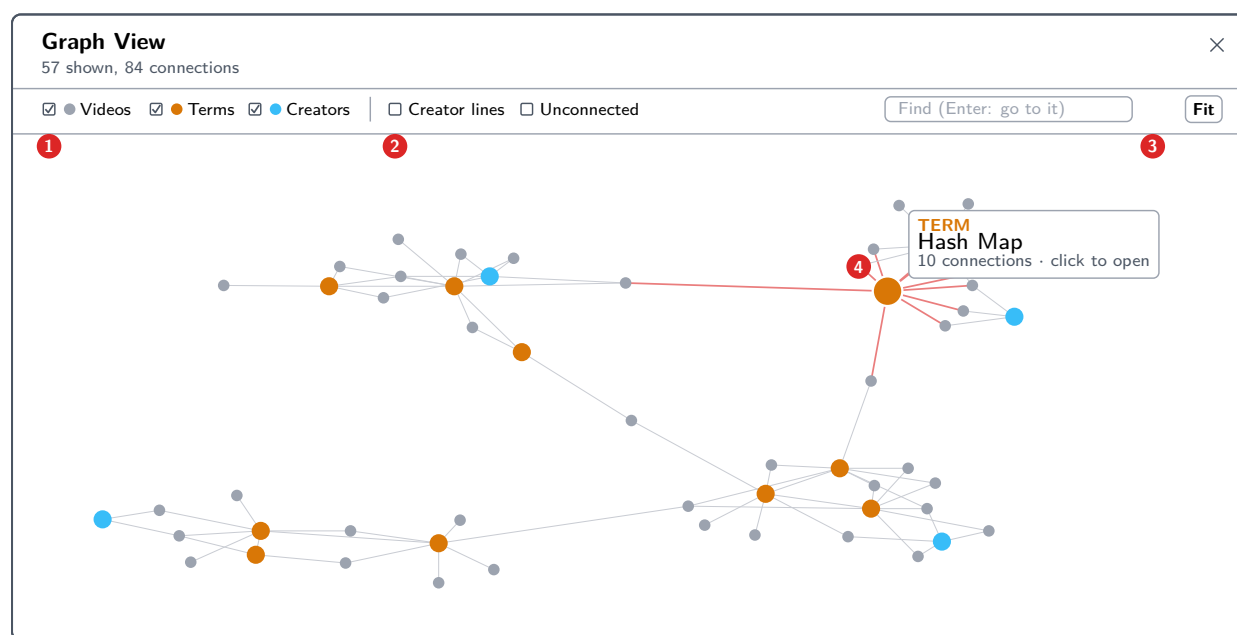


Figure 10: The graph view with a term hovered. Four topics form clusters, joined by the few links between them.

15 Deleting and the Trash

Deleted videos and glossary entries go to the Trash, which is kept in memory and never written to disk. Quitting the app or switching workspace empties it. This keeps deleted data out of sync and exports and stops it from piling up.

A delete takes more than one row with it, so the Trash keeps enough to put everything back: a video's row, its Drive links, its places in sequences, its note, its attachments, its pin, whether it was orphaned, and the links to it that were removed from other texts. Restoring something that has since been created again is refused. The Trash has a size limit, and the oldest items are dropped first.

16 Permissions

Every switch a workspace owner can set is a row in the **Settings** table: hiding a section or settings tab, turning off editing of summaries, Drives, glossary entries and so on. A missing or unreadable value uses its default, so a typo never turns a feature off by surprise. [docs/customizing.md](#) lists them all.

`workspaceReadOnly` turns off every content-editing permission at once without changing their individual values, so turning it off again restores the earlier setup. Read-only still allows what belongs to the reader rather than the workspace: checking off Calendar items and marking them in progress, adding and removing the five Essential tags, pinning, and the Show Alias switch. The structure an owner built (tracks, custom tags, Drives, aliases) stays locked. Catching a renamed channel up with YouTube still happens in a read-only workspace, without the alias prompt, so a reader's copy does not drift from YouTube.

These checks are made in the interface. The backend has no read-only check of its own; a workspace that must not change should be distributed through a sync server, which can enforce settings.

17 Sharing and export

17.1 Kinpak

A Kinpak (`.kinpak`) is a workspace in one file: gzip-compressed newline-delimited JSON, using the same item format as the sync protocol. It starts with a header and the workspace's name, so the importer can show the name without reading the whole file. Then come the Drive tree, biographies, videos and their links, sequences, glossary, custom prompts, notes, attachments, search history, Calendar tracks, and finally the allowed settings.

- API keys, tokens, sync server details and folder paths are never written.
- Attachments are streamed in 4 MiB chunks in both directions, so no file has to fit in memory, and every file is checked against its hash on import.
- The file is written as `name.kinpak.part` and renamed when complete. A file with the real name is always a whole Kinpak.
- Drives can be left out. A plan worked out before writing decides what happens to everything that touches a left-out Drive: its videos, terms filed only there, creators with no remaining videos, and links pointing at any of these, which become plain text.

Importing into a workspace with the same name adds to it and never replaces the reader's own data. A note already written stays. A Calendar track whose name is already taken, or whose time would clash, is skipped and listed under "Left as they were" on the import screen. Calendar progress (what has been checked off) is a separate option, so a teacher can share a course without their own checkmarks. Pins are never included.

17.2 Obsidian

The Obsidian export writes a vault folder: one note per video in folders that mirror the Drive tree, one per glossary entry, one per biography, and one per Calendar track with its entries as a checklist. In-app links become wiki links, and a link to something left out of the export keeps only its text. The export always writes a new folder and never merges into an existing one.

18 Sync

The sync protocol (`docs/sync-protocol.md`) lets a server publish a curated library to many readers. Clients only pull. They ask for a manifest, then page through changes since their last revision. Each server reports an **epoch**; a client whose saved cursor belongs to another epoch does a full resync, because a rebuilt server's revision numbers can be higher than an old cursor and would otherwise look like a normal update.

A row is owned by the server if **SyncItems** has an entry for it. Only owned rows are ever deleted by a sync. When fields are merged, a non-null server value wins and a null one never erases a local value. Videos from a sync skip the normal save path, because its post-save steps rewrite content in ways that are not safe to repeat.

A server can also provide a license for Venice, YouTube and Pixabay. The client never receives the keys. It sends the provider request to the server's proxy with its sync token, and the server adds the key. In **fallback** mode a user's own key wins; in **enforce** mode the proxy is always used.

The reference server serves a read-only copy of a master database. The planned hosted setup uses Turso rather than a self-managed server, with separate license keys for regular readers and collaborators. That work has not started.

19 Interface conventions

A few conventions are applied throughout and are worth keeping when adding screens:

- Every main section has a bottom status bar.
- Dialogs use the shared **Modal** component, and closing one with unsaved changes asks first through the shared **Unsaved Changes** dialog.
- Common controls come from `src/components/ui`: **SecondaryButton** (a dialog or toolbar size), **Select** (a dropdown with its own chevron, since the browser's arrow ignores padding; every dropdown in the app uses it, in one of four looks) and **Segmented** (joined choices, such as Term and Definition in Flashcards). Classes passed to them are merged with **tailwind-merge**, so an override wins whatever order the CSS is in. Writing their classes out by hand is how small differences between screens creep in.
- Permanent deletes ask for confirmation unless the workspace turns that off.
- Errors are shown inline or as a toast, never with a browser `alert()`.
- Colors come from theme variables, so every theme, light ones included, applies everywhere. The accent color marks what is selected or active; it is used sparingly.

- Back and Forward (Alt+Left and Alt+Right, or the mouse's side buttons) move through places visited: a section, a Drive, the video in the sidebar, and the graph view. Typing in a search box updates the current place instead of adding a new one.

20 Testing

The Rust code has about 330 unit tests, run with `cargo test -lib`. Most open a fresh temporary database, so they test real SQL. The production schema tests build a database from `production_schema.sql` and run the app against it end to end. The sync engine is tested against the real reference server.

The TypeScript side is checked with `npx tsc -p tsconfig.app.json --noEmit`. Running `tsc -p .` checks nothing, because the root configuration only references the others.

21 Open work

The main piece of planned work is the hosted sync service described in the Sync section.